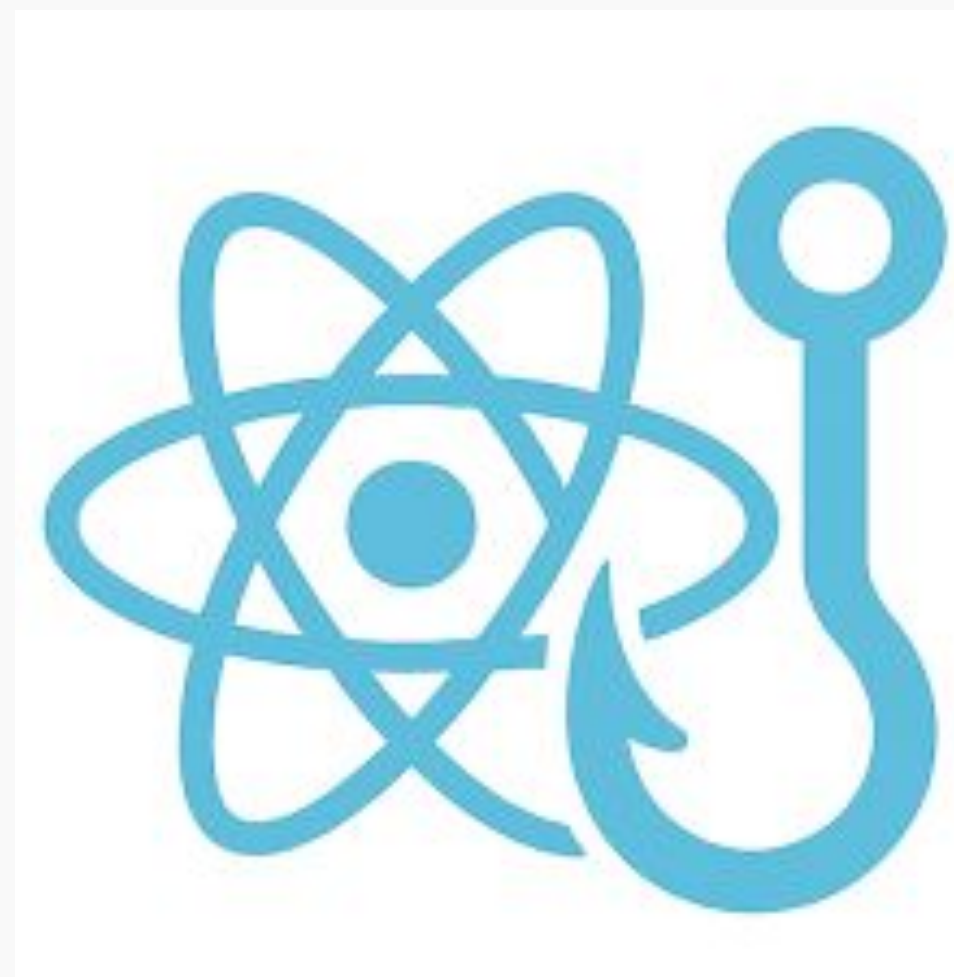


Google Developer Group
Universitas Sriwijaya

Learning 10

Custom Hooks & Reusable Logic

Abstraction and reuse of component logic



```
const filterByOrg = (study, filterByOrg) => filterByOrg ? study.lead_organization === filterByOrg : true  
const filterByStatus = (study, filterByStatus) => filterByStatus ? study.status === filterByStatus : true  
const filterStudies = (studies, filterByOrg, filterByStatus) => studies.filter(study => filterByOrg(study, filterByOrg) && filterByStatus(study, filterByStatus))
```

```
function filterStudies({ studies, filterByOrg, filterByStatus }) {  
  return studies.filter(study => filterByOrg(study, filterByOrg) && filterByStatus(study, filterByStatus))  
}
```

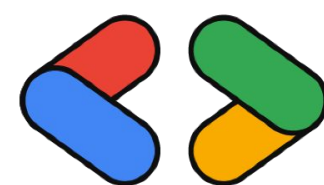
The Problem: Logic Inside Components

As applications grow, components often contain too much logic.

Example problems:

- Fetching data
- Managing loading state
- Handling errors
- UI rendering

```
function StudentsPage() {  
  const [students, setStudents] = useState([])  
  const [loading, setLoading] = useState(true)  
}
```



Separating Logic from UI

Two responsibilities:

UI Layer

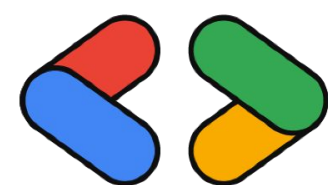
- rendering components
- layout
- styling

Logic Layer

- data fetching
- state management
- side effects

Goal

- keep components focused on UI



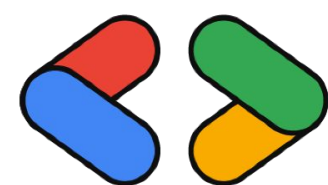
What is a Custom Hook?

A Custom Hook is a function that uses React hooks.

Rules:

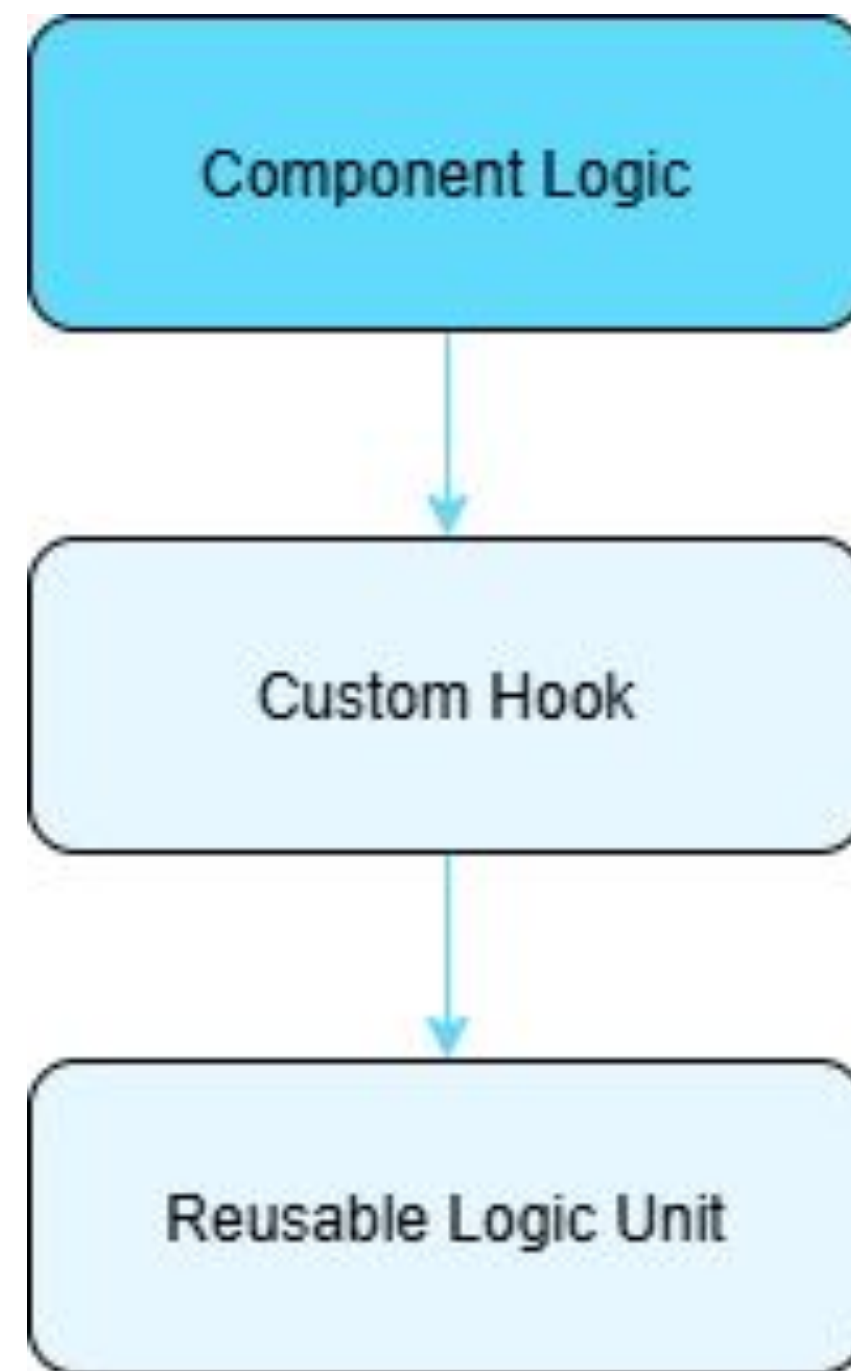
- Function name starts with use
- Can use built-in hooks (useState, useEffect)
- Returns data or functions

```
function useStudents() {  
  const [students, setStudents] = useState([])  
}
```



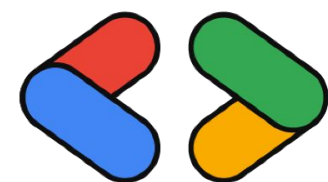
Custom Hook Mental Model

Think of a custom hook as:



Benefits:

- reuse logic
- cleaner components
- easier to maintain



Extracting Logic into a Custom Hook

Before:

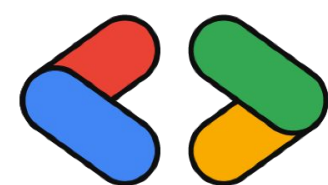
```
useEffect(() => {  
  fetchStudents()  
}, [])
```

Now the component focuses only on:

- rendering UI
- displaying data

After:

```
const { students, loading, error } = useStudents()
```

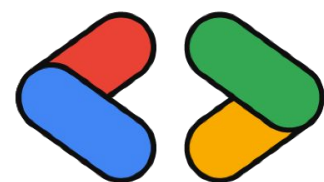


Example Custom Hook

Inside the hook we can manage:

- fetching logic
- loading state
- error handling

```
function useStudents() {  
  const [students, setStudents] = useState([])  
  
  useEffect(() => {  
    loadStudents()  
  }, [])  
}
```



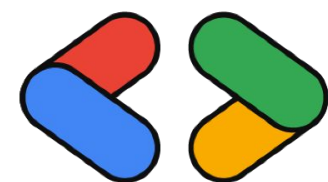
Using the Custom Hook

In the components

```
const { students, loading, error } = useStudents()
```

Component becomes simpler:

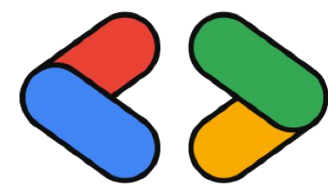
- no fetching logic
- no side effects
- only UI rendering



When Should We Use Custom Hooks?

Use custom hooks when:

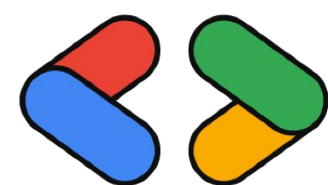
- logic is reused in multiple components
- components become too complex
- logic is unrelated to UI



Challenge! (FE Member Only*)

1. Lakukan refactor ke project kamu di pertemuan 9 sebelumnya, implementasikan custom hooks untuk memisahkan logic dan UI
2. Lakukan pull request ke Frontend Development GDGoC UNSRI 25/26 untuk memasukkan link repo github anda di folder /10-custom-hooks/task.txt

Selamat Mencoba ~



Google Developer Group
Universitas Sriwijaya



Thank You !